



Engineering & Computer Graphics



Lecture 4 — Texture and Sampling

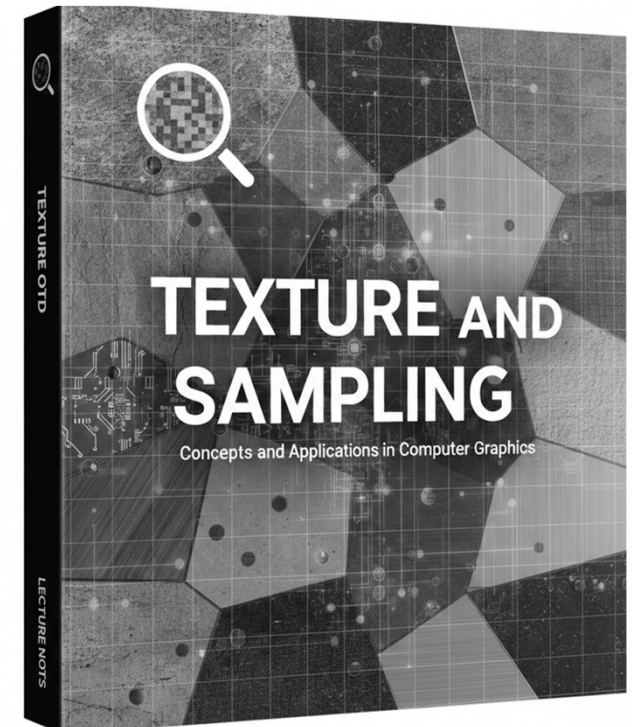
Focus:

- texture mapping,
- interpolation,
- mipmaps,
- aliasing,
- practice

Lecturer: Van-Tien Giap, Ph.D.

tiengv@vnu.edu.vn +84865276899

Faculty of engineering and technology



Outcomes & lecture roadmap

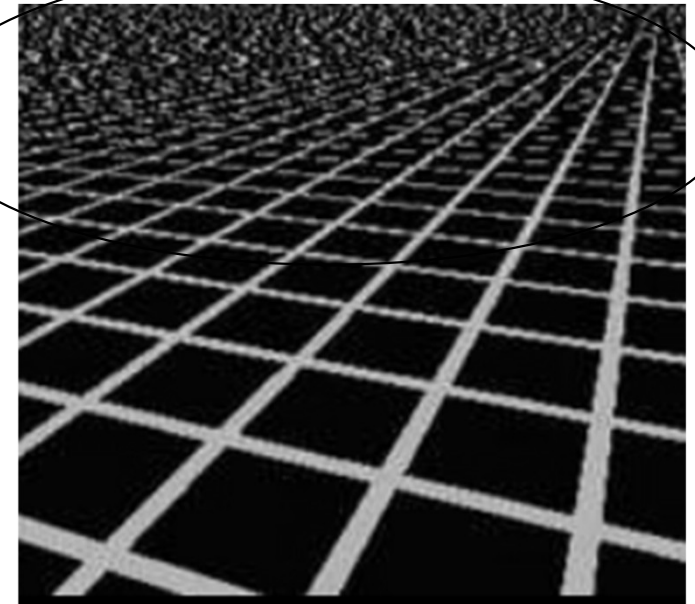
- Explain core concepts: interpolation, mipmaps, aliasing, and pixel footprints
- Design texture coordinates (UV maps) and reason about distortion & seams
- Apply practical filtering: bilinear, trilinear mipmapping, anisotropic filtering
- Practice: implement the texture mapping code and apply texture mapping along with lighting

Why textures and sampling matter

- Textures add detail—but detail + undersampling causes aliasing (jaggies, moiré, shimmer)
- Rendering is *sampling* a continuous signal (surface appearance) onto pixels
- Texture mapping is a resampling problem, like image scaling: minification & magnification
- Goal: get stable images in motion using filtering and good sampling patterns

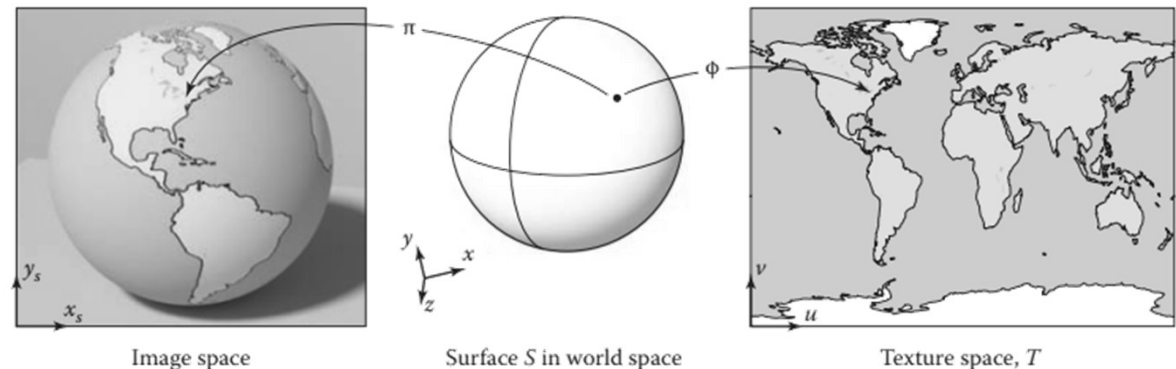
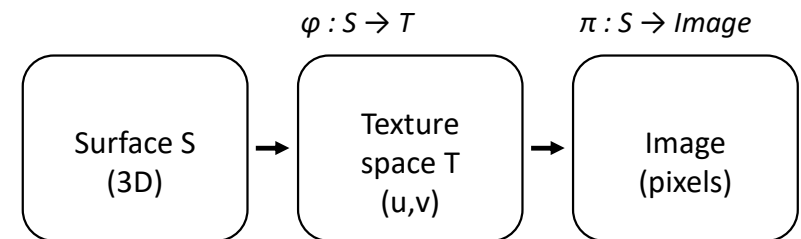
Key idea: aliasing is not a “bug” in the texture—it is a mismatch between signal frequency and sampling rate.

aliasing



Texture mapping pipeline (what is being computed?)

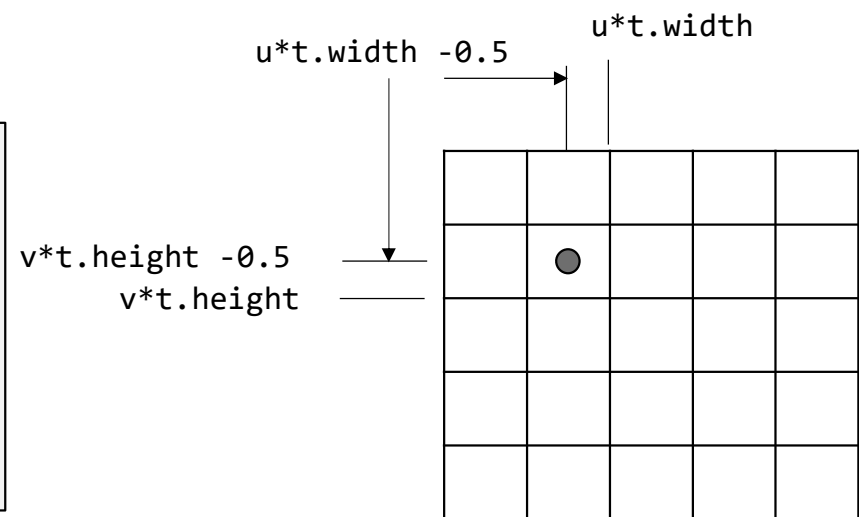
- At each shaded point, compute texture coordinates (u,v) via a texture coordinate function ϕ
- Lookup texel(s) in the texture image to get a texture sample (color or data)
- Combine with shading model (e.g., Lambertian diffuse color, roughness, normals)
- Two big challenges:
 - (1) define ϕ well,
 - (2) filter lookups to control aliasing



Texture lookup basics: texels and nearest sampling

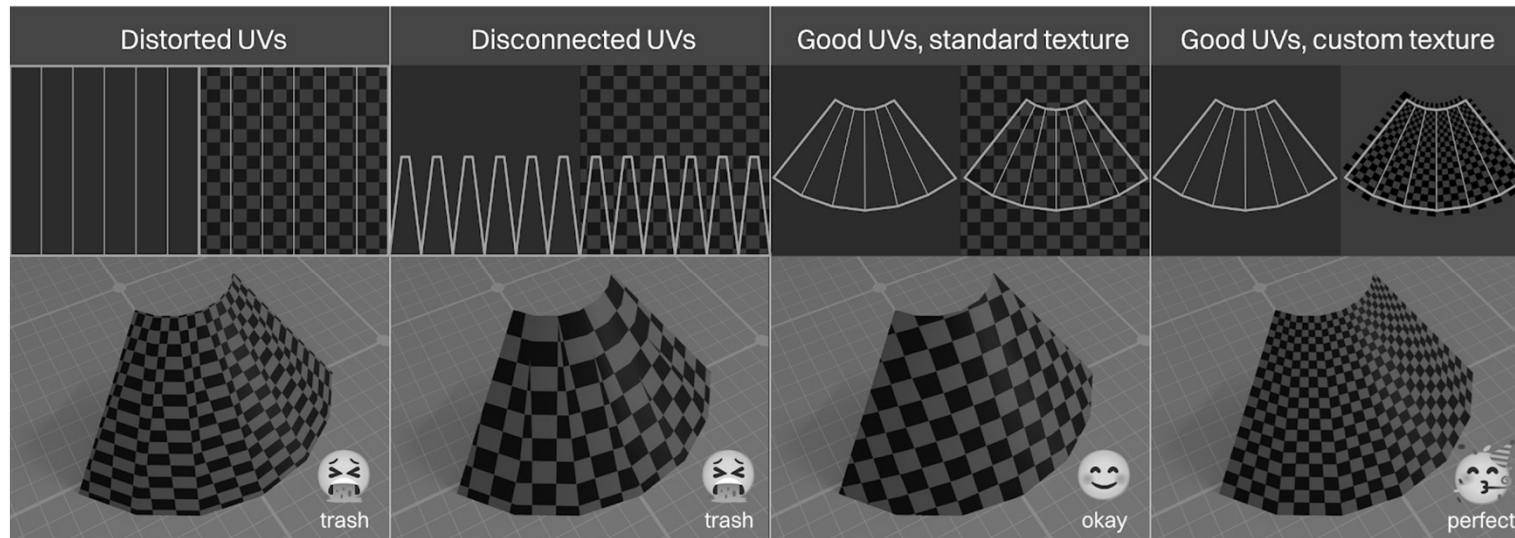
- A texture is usually an image; each pixel in the texture is a *texel*
- Nearest-neighbor lookup: choose closest texel to (u,v) (fast but blocky/aliased)
- Mapping from (u,v) in $[0,1]^2$ to texel indices uses image resolution (width, height)
- In practice, the lookup pattern is irregular but locally coherent → caching matters

```
1 Color tex_lookup(Texture t, float u, float v) {  
2     int i = round(u * t.width - 0.5);  
3     int j = round(v * t.height - 0.5);  
4     return t.get_pixel(i, j);  
5 }
```



Texture coordinate functions (UV mapping): what “good” means

- **Bijectivity**: ideally one-to-one mapping (unless intentionally repeating patterns)
- **Size distortion**: local scale shouldn't vary wildly across the surface (consistent proportional ratio)
- **Shape distortion**: avoid extreme stretching/skewing of local neighborhoods (derivative of ϕ similar in different directions)
- **Continuity**: minimize seams; place unavoidable seams in inconspicuous areas
- *For geometry by equation*: $f(u,v) = (x,y,z)$ just invert the function: $v(u,v) = f^{-1}(x,y,z)$ result might be good
- *For meshes*: you can compute coordinates geometrically or store UVs at vertices



Geometric texture coordinates: planar & projective

- Planar projection: treat texturing like orthographic projection using a matrix M_t
- Works well for mostly-flat surfaces; fails on closed objects (front/back overlap)
- Projective texture coordinates: like perspective projection (matrix P_t)
- Projective mapping is central to shadow mapping (light as a “projector”)

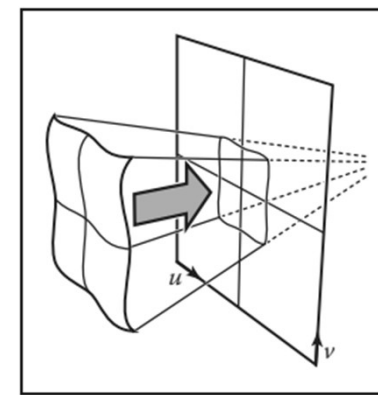
Rule of thumb: if the projection direction becomes tangent to the surface, distortion explodes.

$$\phi(x, y, z) = (u, v) \text{ where } \begin{bmatrix} u \\ v \\ * \\ 1 \end{bmatrix} = M_t \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Orthographic projection



Perspective projection (change M_t to P_t)



Geometric mappings: spherical and cylindrical

- Spherical mapping (latitude/longitude): simple, but high distortion near poles
- Cylindrical mapping: better for column-like shapes; reduces pole-style distortion
- Both introduce a **seam** where atan2 wraps from $+\pi$ to $-\pi$
- When shapes deviate from ideal sphere/cylinder, distortion varies with distance to center/axis

Sphere map



Cylinder map



$$\phi(x, y, z) = ([\pi + \text{atan2}(y, x)]/2\pi, [\pi - \text{acos}(z/\|x\|)]/\pi)$$

$$\phi(x, y, z) = (\frac{1}{2\pi}[\pi + \text{atan2}(y, x)]/2\pi, \frac{1}{2}[1 + z])$$

Cubemaps: reduce polar distortion (more seams, less stretching)

- Instead of one rectangle, project directions/points onto 6 cube faces (6 square textures)
- Less distortion than spherical mapping; seams occur along cube edges
- Face choice: use the coordinate with the largest absolute value ($|x|, |y|, |z|$)
- Widely used for environment maps (reflection mapping) in interactive graphics

$$\phi_{-x}(x, y, z) = \frac{1}{2} [1 + (+z, -y) / |x|],$$

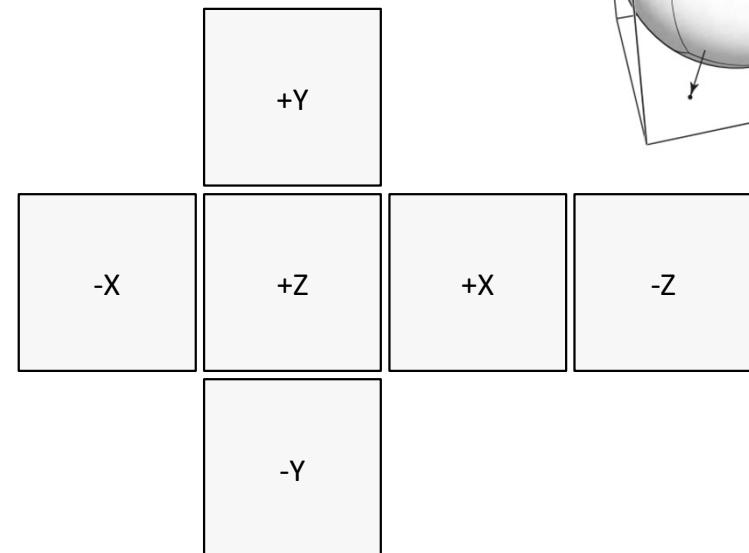
$$\phi_{+x}(x, y, z) = \frac{1}{2} [1 + (-z, -y) / |x|],$$

$$\phi_{-y}(x, y, z) = \frac{1}{2} [1 + (+x, -z) / |y|],$$

$$\phi_{+y}(x, y, z) = \frac{1}{2} [1 + (+x, +z) / |y|],$$

$$\phi_{-z}(x, y, z) = \frac{1}{2} [1 + (-x, -y) / |z|],$$

$$\phi_{+z}(x, y, z) = \frac{1}{2} [1 + (+x, -y) / |z|].$$

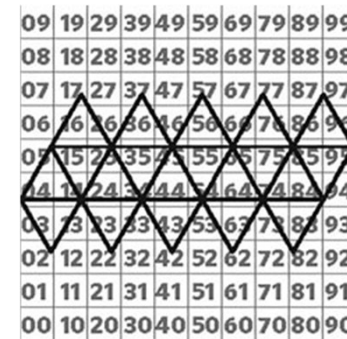


Example cube "net"

Mesh UVs: store at vertices and interpolate across triangles

- For triangle meshes, store (u,v) per vertex and interpolate with **barycentric** coordinates ($u_p = w_A * u_A + w_B * u_B + w_C * u_C$)
- *Continuity* is automatic on shared-vertex meshes (adjacent triangles agree along edges)
- *Injectivity* requires triangles not to overlap in texture space
- *Area distortion*: triangle areas in UV should be proportional to areas on the surface
- *Debug tip*: “unfold” the mesh in texture space and inspect triangle layout

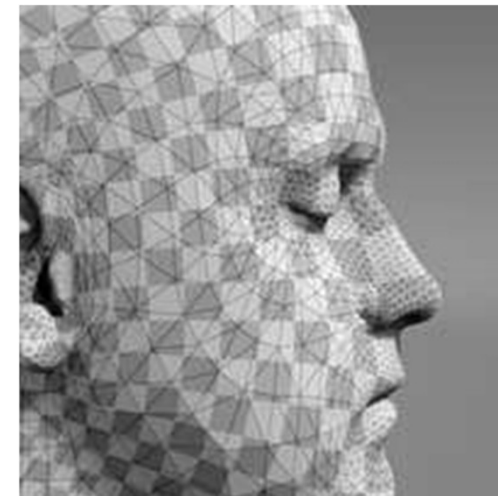
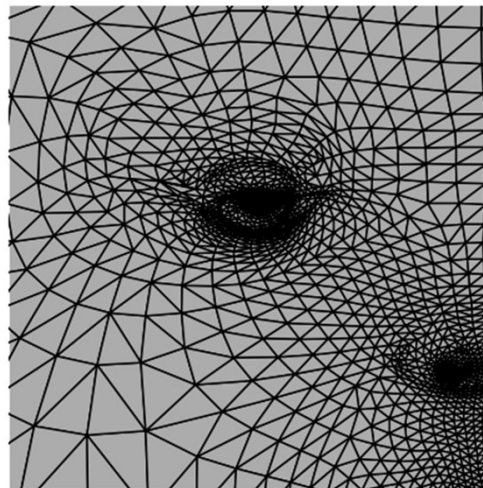
Texture space



Mesh space



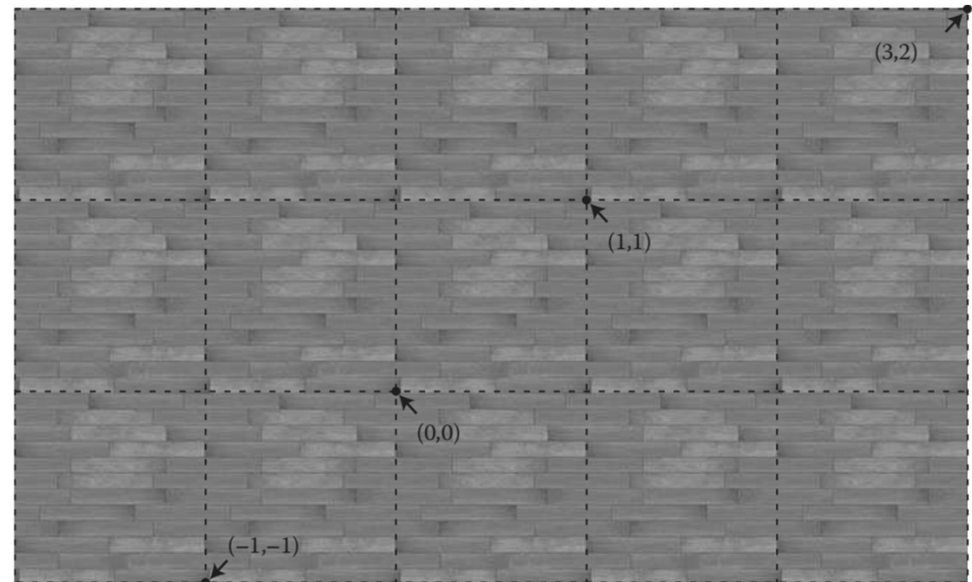
Low shape distortion
But moderate area distortion



Wrapping modes and texture-space transforms

- UVs can go outside $[0,1]^2$: handle with **clamp** (edge color) or **wrap/tile** (repeat)
- Wrap uses remainder/modulo indexing; clamp saturates indices to valid range
- Modelers often adjust scale/offset/rotation using a 2D transform matrix MT
- Think of textures as functions defined over the infinite plane (wrapping makes this explicit)

```
1 // Wrap (tiling)
2 i = (round(u*w - 0.5)) % w;
3 j = (round(v*h - 0.5)) % h;
4
5 // Clamp
6 ic = max(0, min(i, w-1));
7 jc = max(0, min(j, h-1));
```



Perspective-correct interpolation (1/2): the problem

- **Naïve approach:** interpolate (u,v) linearly in screen space → looks wrong in perspective
- **Reason:** perspective projection is non-linear; the world-space interpolation parameter $\neq$ screen-space parameter
- **Symptoms:** textures “swim” or warp on receding surfaces
- **Fix:** interpolate attributes consistently after perspective divide

Transform to screen space

$$\begin{bmatrix} u \\ v \\ 1 \\ x_r \\ y_r \\ z_r \\ w_r \end{bmatrix} \xrightarrow{\text{homogenize}} \begin{bmatrix} u/w_r \\ v/w_r \\ 1/w_r \\ x_r/w_r = x_s \\ y_r/w_r = y_s \\ z_r/w_r = z_s \\ 1 \end{bmatrix}$$

interpolate

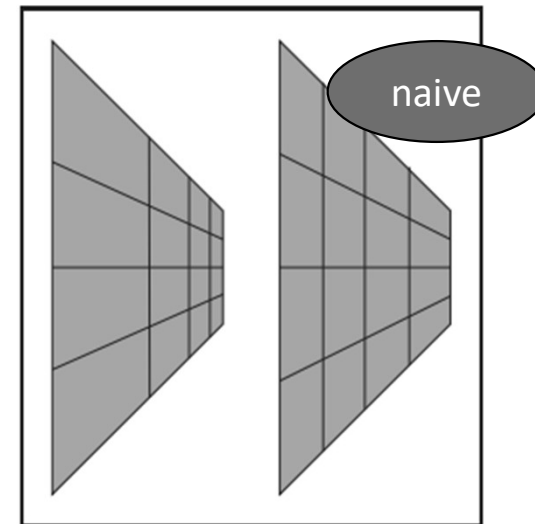
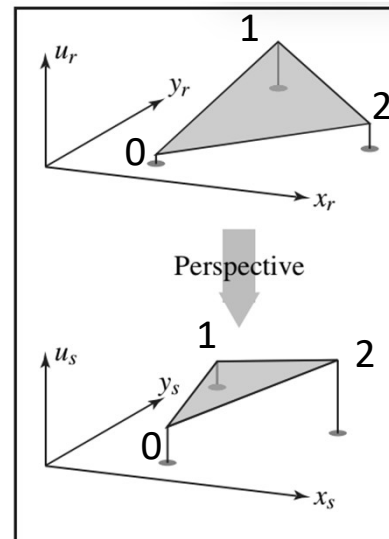
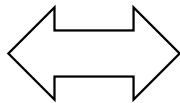


Figure 11.15. Left: correct perspective. Right: interpolation in screen space.

Perspective-correct interpolation (2/2): code

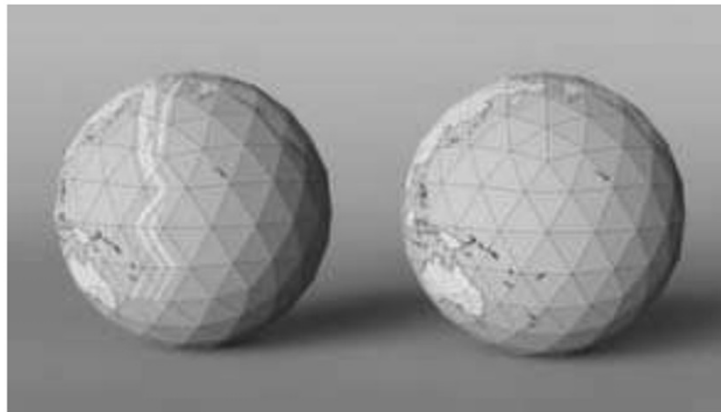
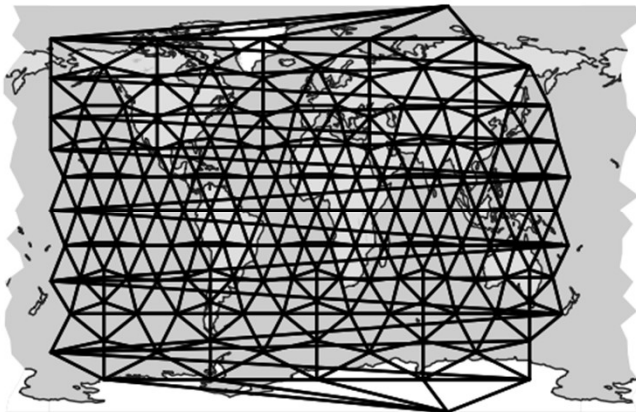
- Carry attributes through the same projective transform as geometry
- Linearly interpolate $u/w \rightarrow u_s$, $v/w \rightarrow v_s$, and $1/w \rightarrow w_s$ in screen space using barycentric weights
- Recover u and v per fragment: $u = u_s/w_s$, $v = v_s/w_s$
- This matches the correct world-space interpolation along projected lines/triangles

```
1 // Given per-vertex (u,v,w) after view-projection but before divide
2 us = a*(u0/w0) + b*(u1/w1) + c*(u2/w2);
3 vs = a*(v0/w0) + b*(v1/w1) + c*(v2/w2);
4 ws = a*(1/w0) + b*(1/w1) + c*(1/w2);
5
6 u = us / ws;
7 v = vs / ws;
```

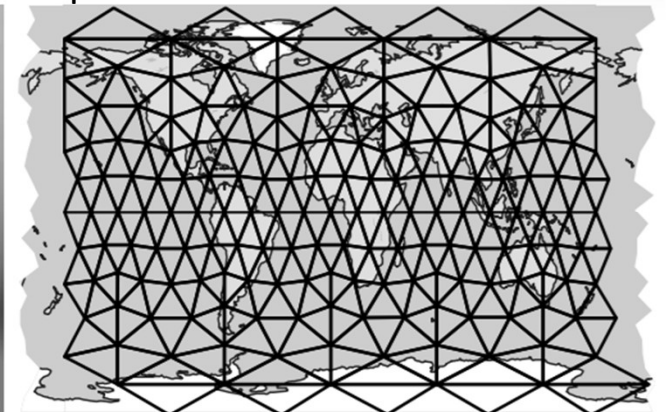
Continuity & seams: why vertices sometimes must be duplicated

- Closed surfaces cannot be mapped bijectively and continuously onto one rectangle (topology)
- Triangles crossing a seam can interpolate the “long way” around texture space
- **Solution:** duplicate vertices at seams (same 3D position, different UVs) instead of sharing texture coordinate
- This allows clean interpolation on each side and prevents extreme compression/distortion

Share texture coordinate at seam

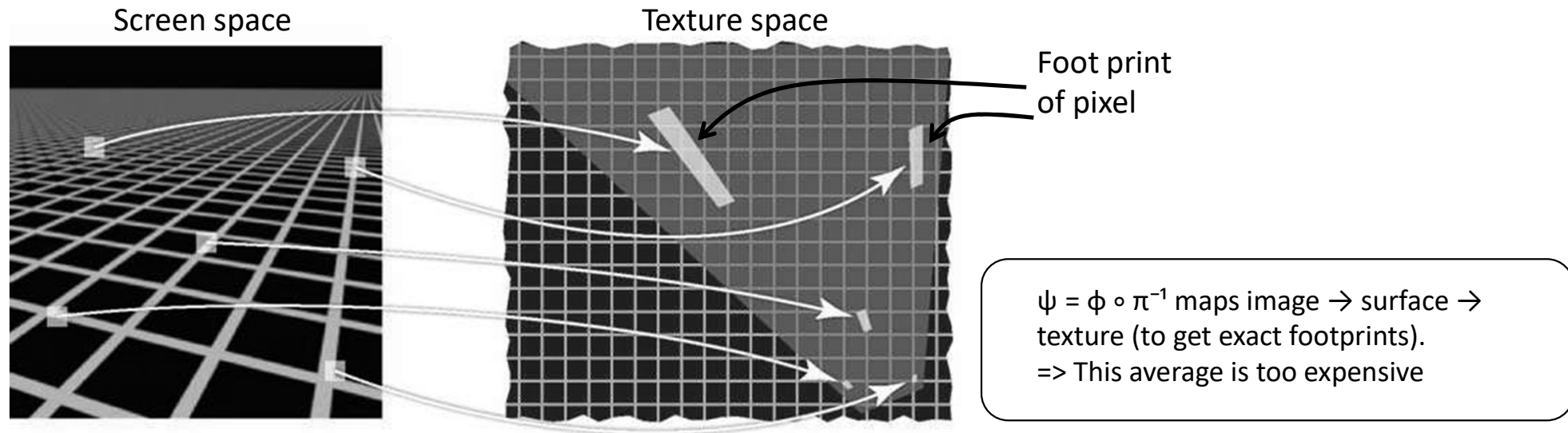


duplicate vertex at seam



Texture antialiasing: pixels have *footprints* in texture space

- A **pixel value** (color) in screen space should be an **area average** over that **pixel's area** in texture space, not a point sample
- Through projection + texture mapping, a pixel area maps to a footprint in texture space
- Footprints vary across the image (distance, view angle) and depend on ϕ distortions
- **Goal:** compute filtered texture sample $\approx$ average of texture over the footprint



Footprint approximation using derivatives (Jacobian J)

- Approximate ψ locally as linear:

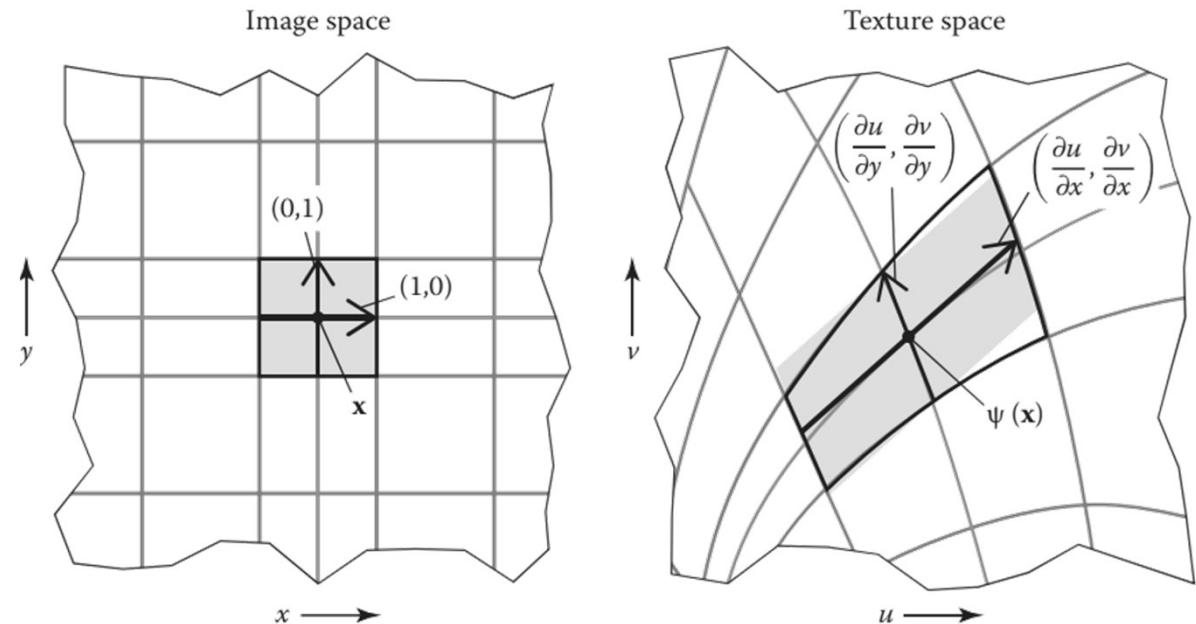
$$\psi(\mathbf{x}) = \psi(\mathbf{x}_0) + J(\mathbf{x} - \mathbf{x}_0)$$

- J contains partial derivatives du/dx , du/dy , dv/dx , dv/dy
- A unit pixel square maps to a parallelogram spanned by vectors u_x and u_y

$$u_x = (du/dx, dv/dx) \text{ and}$$

$$u_y = (du/dy, dv/dy)$$

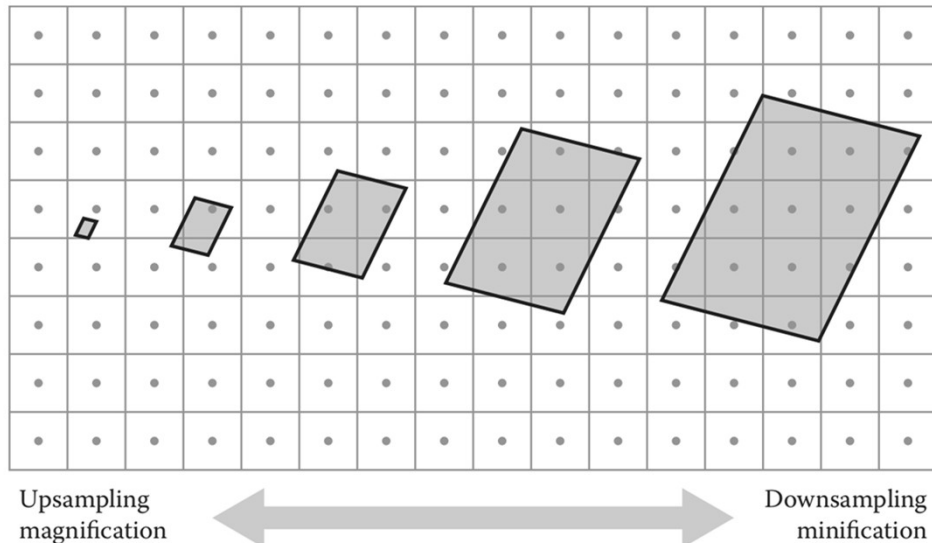
- Large derivatives $\rightarrow$ large footprints; skew/unequal lengths $\rightarrow$ anisotropic footprints



Quality vs speed: many filters approximate the true footprint average differently.

Reconstruction (magnification): bilinear interpolation

- When footprint < 1 texel, we magnify: need smooth reconstruction between texels
- Bilinear interpolation reads the **4 neighboring texels** and blends by fractional offsets
- High-quality large filters are expensive because texture lookups are irregular
- Hardware texture units accelerate bilinear filtering with dedicated caches



```
1 Color tex_bilinear(Texture t, float u, float v) {  
2 float up = u * t.width - 0.5;  
3 float vp = v * t.height - 0.5;  
4 int left_point = floor(up), right_point = left_point + 1;  
5 int top_point = floor(vp), bot_point = top_point + 1;  
6 float f_left = up - left_point, f_top = vp - top_point;  
7 float f_right = 1 - f_left, f_bot = 1 - f_top;  
8 return f_left*f_top*t[left_point][top_point]  
    + f_left*f_bot*t[left_point][bot_point]  
    + f_right*f_top*t[right_point][top_point]  
    + f_right*f_bot*t[right_point][bot_point]  
9 }
```

Mipmapping (minification): precompute averages at multiple scales

- When footprint covers many texels, we need an average—point sampling aliases badly
- A mipmap is an image pyramid: level 0 is full resolution; each next level is 2× downsampled
- A texel at level k represents an average over roughly a $2^k \times 2^k$ area of level 0
- This makes large-area averaging fast: sample from an appropriate mip level

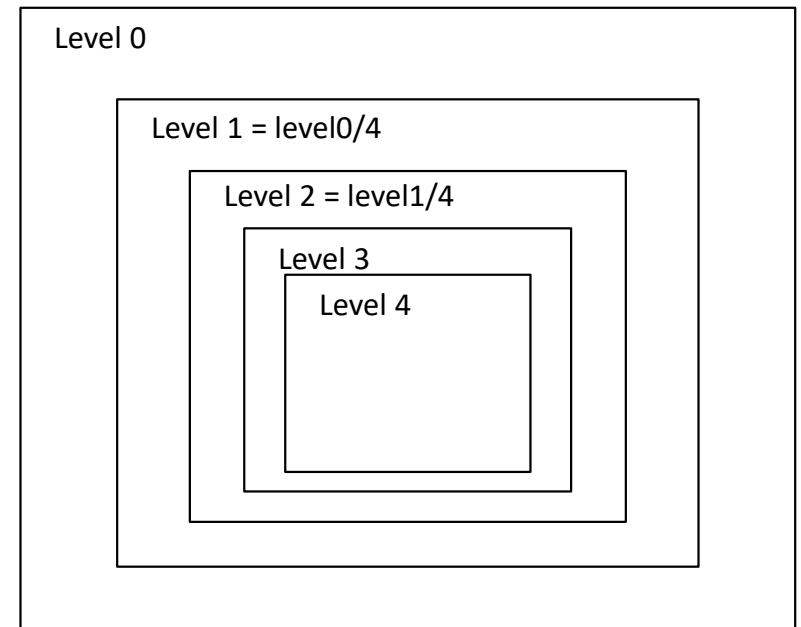


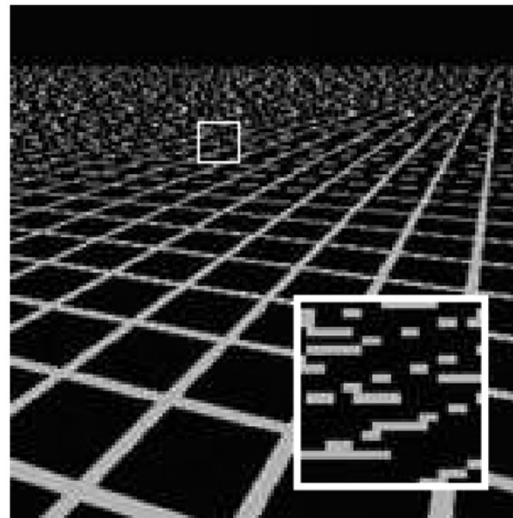
Image pyramid (mipmap)

“Multum in parvo”: much in a small space (why it’s called mip).

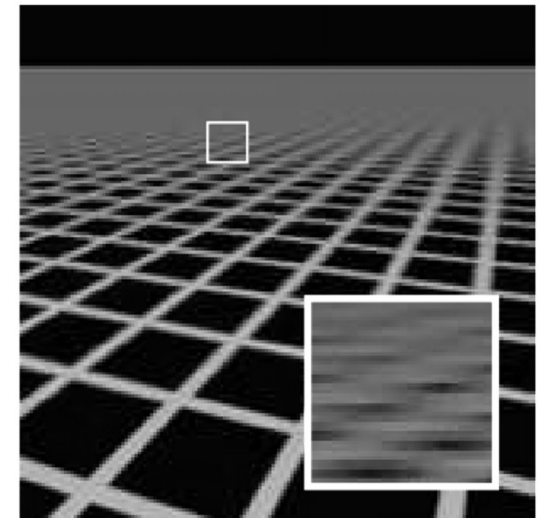
Choosing mip levels: $D \approx \max(|u_x|, |u_y|)$ and trilinear blending

- Estimate footprint width D in texels (common choice: longest edge length)
- Choose level $k \approx \log_2(D)$ (texels at level k cover width 2^k in level 0)
- Nearest-level sampling causes visible transitions; trilinear blends between k_0 and k_1
- Limitation: footprints can be elongated $\rightarrow$ square mip texels can over-blur

```
Color mipmap_sample_trilinear(Texture  
mip[], float u, float v,  
matrix J) {  
    D = max_column_norm(J)  
    k = log2(D)  
    k0 = floor(k); k1 = k0 + 1  
    a = k1 - k; b = 1 - a  
    c0 = tex_sample_bilinear(mip[k0], u, v)  
    c1 = tex_sample_bilinear(mip[k1], u, v)  
    return a*c0 + b*c1  
}
```



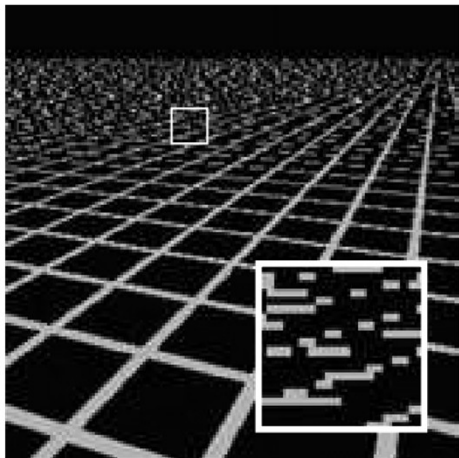
Point sampling



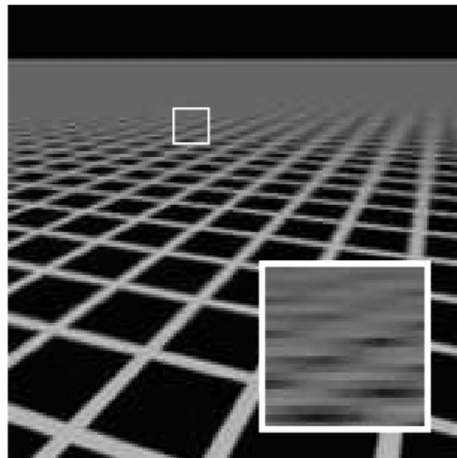
Trilinear mipmap

Anisotropic filtering: handle elongated footprints at grazing angles

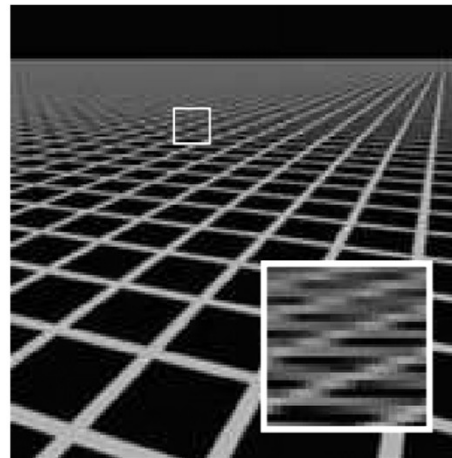
- Grazing angles produce anisotropic footprints (long, thin regions in texture space)
- Basic mipmapping approximates with a square → excessive blur along one direction
- Anisotropic filtering: pick mip level from the *short* axis, then take **multiple taps** along the long axis
- Common in GPUs; improves floor/road textures dramatically in real-time rendering



Point sampling



Trilinear mipmap



Anisotropic filtering

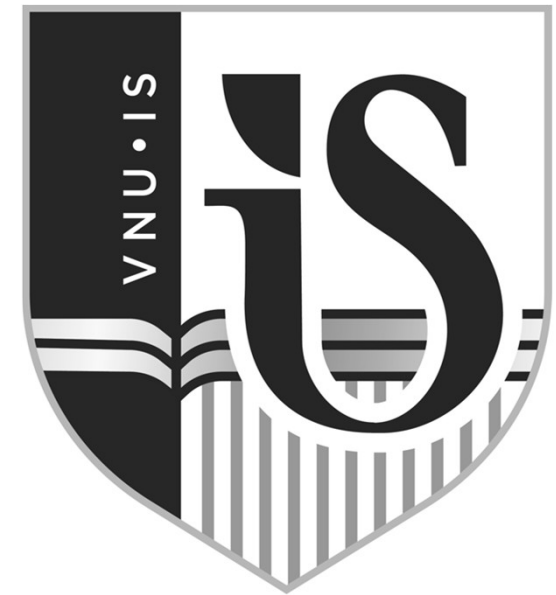
Mental model: “and prefilter based on the short axis, then manually average many lookups along the long axis,”.

Practice texture mapping and lighting

Modify the week 3 project to complete following tasks:

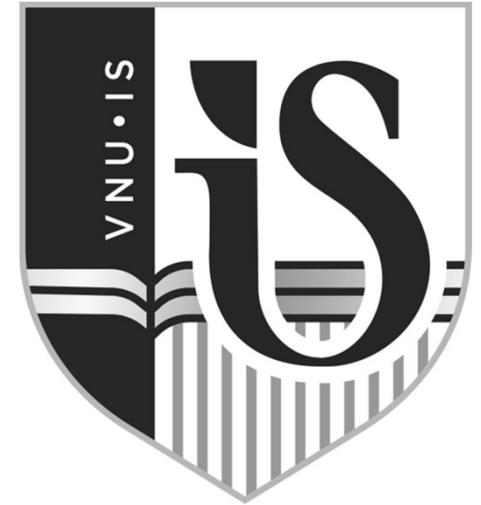
- Task A: render a textured plane with a detail-rich image, nearest filter
- Task B: compare nearest vs bilinear; observe magnification artifacts
- Task C: enable mipmaps + trilinear; observe reduced aliasing in minification
- Task D: enable anisotropic filtering; compare sharpness at grazing angles

Deliverable: 3 screenshots + a short note explaining what changed and why.



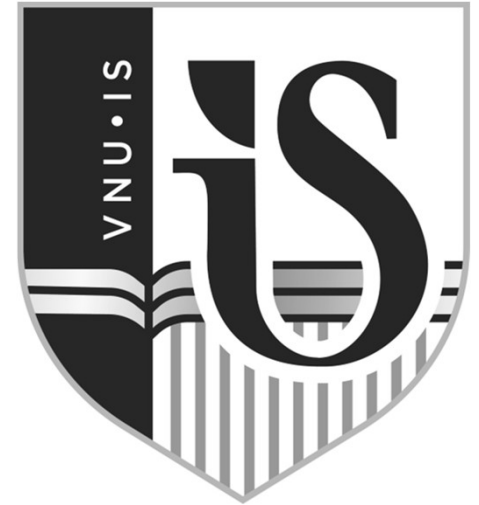
Task A: render a textured plane with a detail-rich image

No.	What	Where
1	stb_image include	Top of file, after existing includes
2	Declare anisotropic enums	After includes
3	loadTexture2D() function	Before main()
4	Cube vertex data (add UVs)	Inside createCubeVAO(), add u,v for each vertex, add buffer
5	Vertex shader (add aTexCoord, vTexCoord)	Insert UVs to kVS_part1 string
6	Fragment shader (add uTexture, sampling)	Insert texture sampling to kFS_part1 string
7	loadTexture(...) call	In main(), after createCubeVAO()
8	glGetUniformLocation(prog, "uTexture")	In main(), with other uniform lookups
9	glActiveTexture + glBindTexture + glUniform1i	In render loop, before glDrawArrays



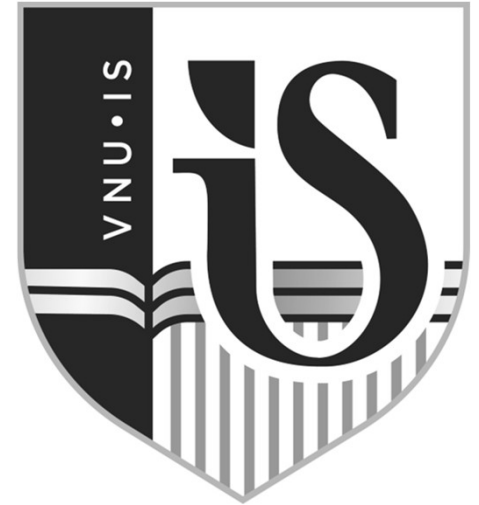
Task A: render a textured plane with a detail-rich image

No.	What	How
1	stb_image include	<pre>#define STB_IMAGE_IMPLEMENTATION #include "stb_image.h"</pre>
2	Declare anisotropic enums	<pre>#ifndef GL_TEXTURE_MAX_ANISOTROPY #define GL_TEXTURE_MAX_ANISOTROPY 0x84FE #endif #ifndef GL_MAX_TEXTURE_MAX_ANISOTROPY #define GL_MAX_TEXTURE_MAX_ANISOTROPY 0x84FF #endif</pre>



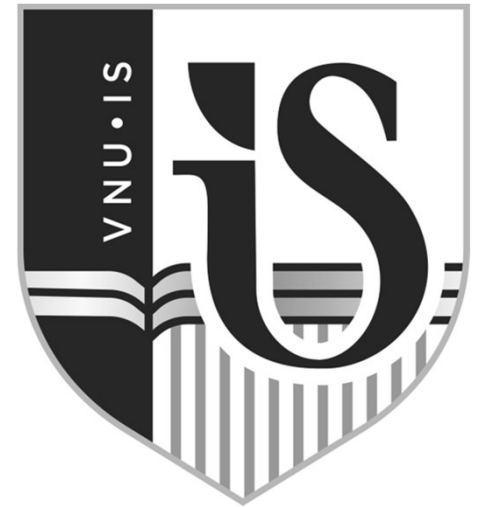
Task A: render a textured plane with a detail-rich image

No.	What	How
3	loadTexture2D function definition	<pre>static GLuint loadTexture2D(const char* path) { //declare and bind tex variable GLuint tex; glGenTextures(1, &tex); glBindTexture(GL_TEXTURE_2D, tex); // --- Wrapping --- glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT); glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT); // --- Filtering --- // Minification: nearest glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST); // Magnification: nearest glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST); }</pre>



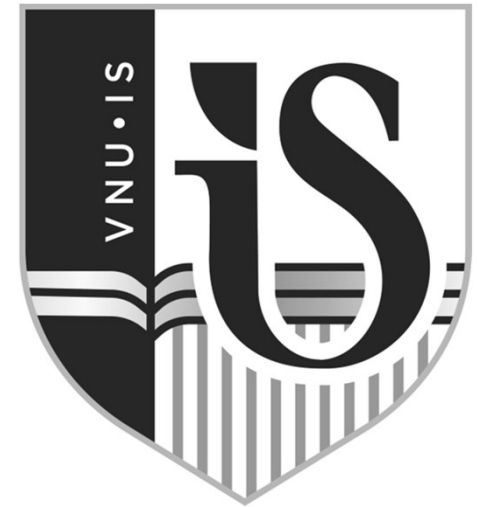
Task A: render a textured plane with a detail-rich image

No.	What	How
3	loadTexture2D function defition	<pre>//(cont.) // --- Load image --- int width, height, channels; stbi_set_flip_vertically_on_load(true); // OpenGL UV origin is bottom-left unsigned char* data = stbi_load(path, &width, &height, &channels, 0); if (!data) { std::cerr << "Failed to load texture: " << path << "\n"; return 0; } GLenum fmt = (channels == 4) ? GL_RGBA : GL_RGB; glTexImage2D(GL_TEXTURE_2D, 0, fmt, width, height, 0, fmt, GL_UNSIGNED_BYTE, data); // --- Auto-generate mipmaps --- // This creates all mip levels needed for trilinear filtering glGenerateMipmap(GL_TEXTURE_2D); stbi_image_free(data); glBindTexture(GL_TEXTURE_2D, 0); return tex; }</pre>



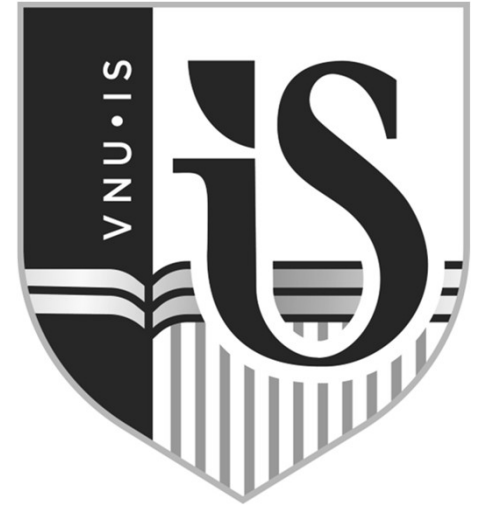
Task A: render a textured plane with a detail-rich image

No.	What	How
4	Cube vertex data (add UVs)	<pre> static GLuint createCubeVAO() { float v[] = { // pos // normal // UV // +X 1,-1,-1, 1,0,0, 0,0, 1, 1,-1, 1,0,0, 1,0, 1, 1, 1, 1,0,0, 1,1, 1,-1,-1, 1,0,0, 0,0, 1, 1, 1, 1,0,0, 1,1, 1,-1, 1, 1,0,0, 0,1, // -X -1,-1, 1, -1,0,0, 0,0, -1, 1, 1, -1,0,0, 1,0, -1, 1,-1, -1,0,0, 1,1, -1,-1, 1, -1,0,0, 0,0, -1, 1,-1, -1,0,0, 1,1, -1,-1,-1, -1,0,0, 0,1, // +Y -1, 1,-1, 0,1,0, 0,0, 1, 1,-1, 0,1,0, 1,0, 1, 1, 1, 0,1,0, 1,1, -1, 1,-1, 0,1,0, 0,0, 1, 1, 1, 0,1,0, 1,1, -1, 1, 1, 0,1,0, 0,1, // -Y -1,-1, 1, 0,-1,0, 0,0, 1,-1, 1, 0,-1,0, 1,0, 1,-1,-1, 0,-1,0, 1,1, -1,-1, 1, 0,-1,0, 0,0, 1,-1,-1, 0,-1,0, 1,1, -1,-1,-1, 0,-1,0, 0,1, // +Z 1,-1, 1, 0,0,1, 0,0, 1, 1, 1, 0,0,1, 1,0, -1, 1, 1, 0,0,1, 1,1, 1,-1, 1, 0,0,1, 0,0, -1, 1, 1, 0,0,1, 1,1, -1,-1, 1, 0,0,1, 0,1, // -Z -1,-1,-1, 0,0,-1, 0,0, -1, 1,-1, 0,0,-1, 1,0, 1, 1,-1, 0,0,-1, 1,1, -1,-1,-1, 0,0,-1, 0,0, 1, 1,-1, 0,0,-1, 1,1, 1,-1,-1, 0,0,-1, 0,1, }; </pre>



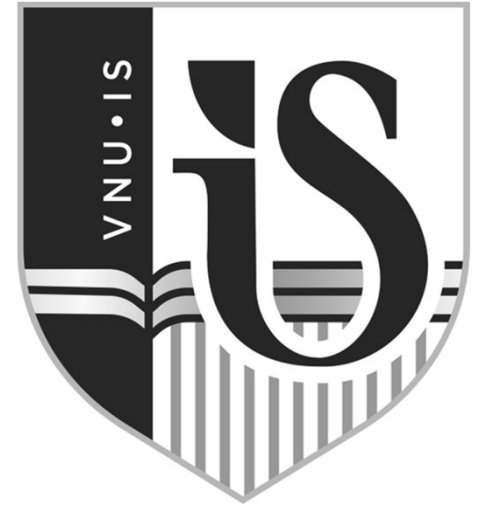
Task A: render a textured plane with a detail-rich image

No.	What	How
4	Cube vertex data (add UVs)	<pre>//cont. GLuint vao=0, vbo=0; glGenVertexArrays(1, &vao); glGenBuffers(1, &vbo); glBindVertexArray(vao); glBindBuffer(GL_ARRAY_BUFFER, vbo); glBufferData(GL_ARRAY_BUFFER, sizeof(v), v, GL_STATIC_DRAW); // location 0: position (3 floats) glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 8*sizeof(float), (void*)0); glEnableVertexAttribArray(0); // location 1: normal (3 floats) glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 8*sizeof(float), (void*)(3*sizeof(float))); glEnableVertexAttribArray(1); // location 2: UV (2 floats) <-- NEW glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, 8*sizeof(float), (void*)(6*sizeof(float))); glEnableVertexAttribArray(2); glBindVertexArray(0); return vao; }</pre>



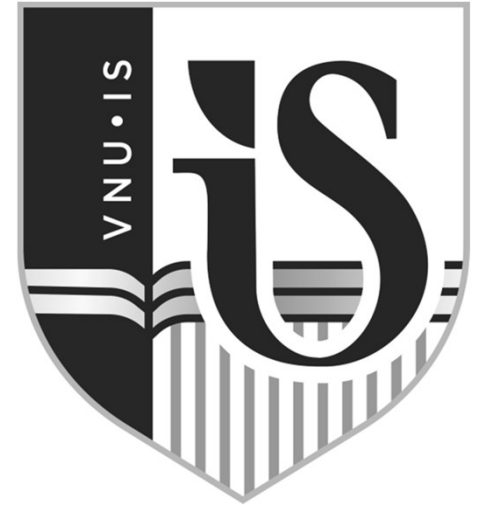
Task A: render a textured plane with a detail-rich image

No.	What	How
5	Vertex shader (add aTexCoord, vTexCoord)	<pre> static const char* kVS_part1 = R"GLSL(#version 330 core layout(location=0) in vec3 aPos; layout(location=1) in vec3 aNormal; layout(location=2) in vec2 aTexCoord; // NEW uniform mat4 uM; uniform mat4 uV; uniform mat4 uP; uniform mat3 uNormalMat; out vec3 vWorldPos; out vec3 vWorldNormal; out vec2 vTexCoord; // NEW void main() { vec4 wp = uM * vec4(aPos, 1.0); vWorldPos = wp.xyz; vWorldNormal = normalize(uNormalMat * aNormal); vTexCoord = aTexCoord; // NEW gl_Position = uP * uV * wp; })GLSL"; </pre>



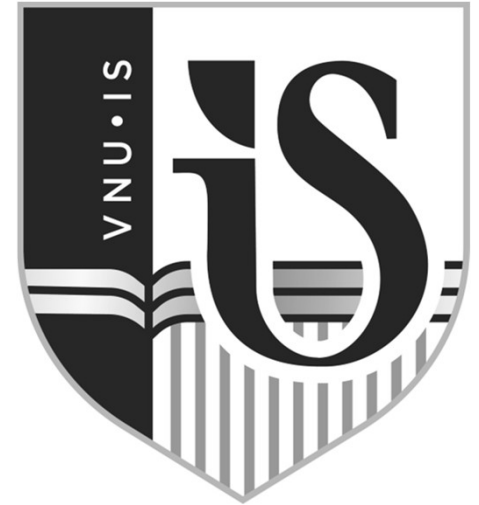
Task A: render a textured plane with a detail-rich image

No.	What	How
6	Fragment shader (add uTexture, sampling)	<pre>static const char* kFS_part1 = R"GLSL(#version 330 core in vec3 vWorldPos; in vec3 vWorldNormal; in vec2 vTexCoord; // NEW out vec4 FragColor; uniform sampler2D uTexture; // NEW uniform vec3 uViewPos; uniform vec3 uLightPos; uniform vec3 uLightColor; uniform vec3 uAmbientColor; uniform vec3 uKa; uniform vec3 uKd; uniform vec3 uKs; uniform float uShininess; uniform bool uUseBlinn; uniform bool uDebugNormals;</pre>



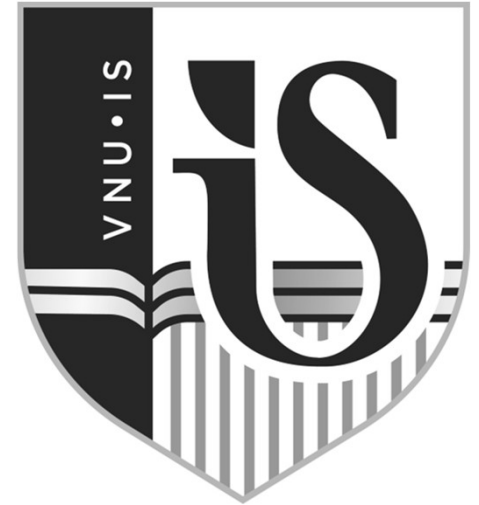
Task A: render a textured plane with a detail-rich image

No.	What	How
6	Fragment shader (add uTexture, sampling)	<pre> void main() { vec3 N = normalize(vWorldNormal); if (uDebugNormals) { FragColor = vec4(N * 0.5 + 0.5, 1.0); return; } vec3 texColor = texture(uTexture, vTexCoord).rgb; // NEW: sample texture vec3 L = normalize(uLightPos - vWorldPos); vec3 V = normalize(uViewPos - vWorldPos); vec3 ambient = uKa * uAmbientColor * texColor; // texture modulates ambient (MOD) float ndotL = max(dot(N, L), 0.0); vec3 diffuse = uKd * ndotL * uLightColor * texColor; // texture modulates diffuse (MOD) float spec = 0.0; if (ndotL > 0.0) { if (uUseBlinn) { vec3 H = normalize(L + V); spec = pow(max(dot(N, H), 0.0), uShininess); } else { vec3 R = reflect(-L, N); spec = pow(max(dot(R, V), 0.0), uShininess); } } vec3 specular = uKs * spec * uLightColor; // specular stays white FragColor = vec4(ambient + diffuse + specular, 1.0);})GLSL"; </pre>



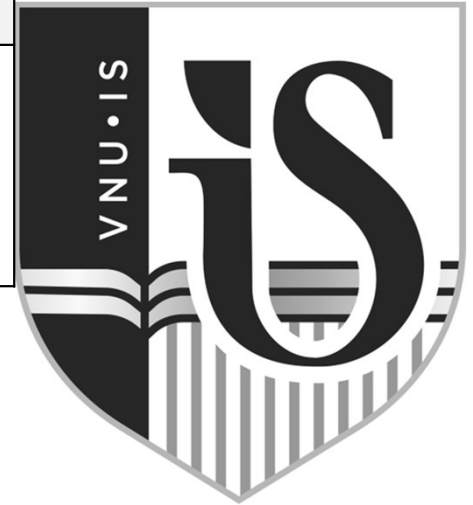
Task A: render a textured plane with a detail-rich image

No.	What	How
7	loadTexture(..) call	<pre>//after declaration of cubeVAO GLuint texture = loadTexture2D("your_image.png"); // put your image path here</pre>
8	connect uTexture var	<pre>//after calling glUseProgram(prog) GLint uTexture = glGetUniformLocation(prog, "uTexture");</pre>
9	Activate, bind and transfer value	<pre>//before glBindVertexArray(cubeVAO); glActiveTexture(GL_TEXTURE0); glBindTexture(GL_TEXTURE_2D, texture); glUniform1i(uTexture, 0); // tell the shader to use texture unit 0</pre>



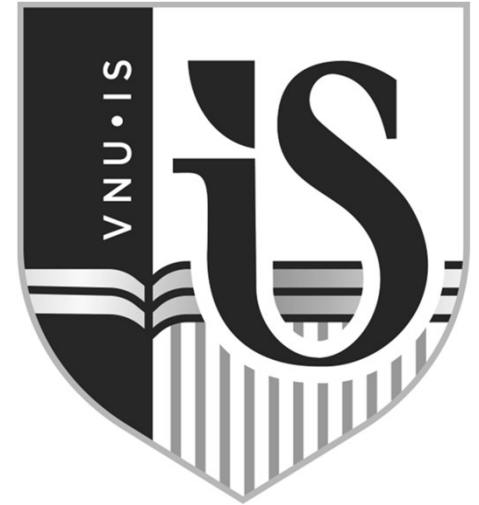
Task B: compare nearest vs bilinear; observe magnification artifacts

No.	What	How
1	Modify the loadTexture2D function	<pre>// --- Filtering --- modified // Minification: linear (linear interpolation) glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR); // Magnification: linear (no mipmaps used when magnifying) glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);</pre>



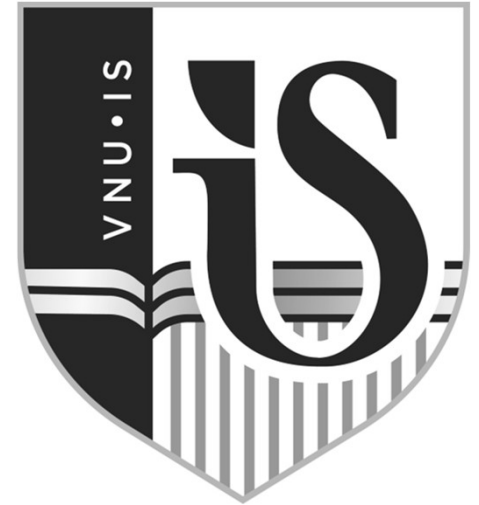
Task C: enable mipmaps + trilinear

No.	What	How
1	Modify the loadTexture2D function	<pre>// --- Filtering --- modified // Minification: trilinear (linear mipmap interpolation + linear within mip) glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR); // Magnification: bilinear (no mipmaps used when magnifying) glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);</pre>



Task D: enable anisotropic filtering; compare sharpness at grazing angles

No.	What	How
1	Modify the loadTexture2D function	<pre>//adding this // --- Anisotropic filtering --- float maxAniso = 1.0f; //initiating maxAniso level glGetFloatv(GL_MAX_TEXTURE_MAX_ANISOTROPY, &maxAniso); //get max allowable Aniso level glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAX_ANISOTROPY, maxAniso);</pre>



Wrap-up: what you should be able to do after this lecture

- Explain aliasing and how filtering (bilinear, mipmaps, anisotropic) reduces it
- Design/diagnose UV mappings: distortion, wrapping modes, seams, perspective correctness
- Compute pixel footprints conceptually using derivatives; pick an appropriate filter
- implement UV mapping and lighting

Homework:

- 1) Positioning the image on the selected wall
- 2) Import STL file and glue the image on specific position of it.